

SCHOOL OF COMPUTER SCIENCE AND ARTIFICIAL INTELLIGENCE		DEPARTMENT OF COMPUTER SCIENCE ENGINEERING	
Program Name: B. Tech		Assignment Type: Lab	Academic Year:2025-2026
Course Coordinator Name		Venkataramana Veeramsetty	
Instructor(s) Name		Dr. V. Venkataramana (Co-ordinator)	
		Dr. T. Sampath Kumar	
		Dr. Pramoda Patro	
		Dr. Brij Kishor Tiwari	
		Dr.J.Ravichander	
		Dr. Mohammand Ali Shaik	
		Dr. Anirodh Kumar	
		Mr. S.Naresh Kumar	
		Dr. RAJESH VELPULA	
		Mr. Kundhan Kumar	
		Ms. Ch.Rajitha	
		Mr. M Prakash	
		Mr. B.Raju	
		Intern 1 (Dharma teja)	
		Intern 2 (Sai Prasad)	
		Intern 3 (Sowmya)	
		NS_2 (Mounika)	
Course Code	24CS002PC215	Course Title	AI Assisted Coding
Year/Sem	II/I	Regulation	R24
Date and Day of Assignment	Week5 - Monday	Time(s)	
Duration	2 Hours	Applicable to Batches	
AssignmentNumber: 9.1(Present assignment number)/24(Total number of assignments)			
Q.No.	Question		Expected Time to complete
1	Lab 17– AI for Data Processing: Data cleaning and preprocessing scripts The objective of this lab is to enable students to understand and apply AI-assisted coding tools for automating and enhancing data preprocessing tasks. Students will: 1. Gain practical experience in cleaning, transforming, and standardizing real-world datasets with issues such as missing		Week 9- Monday

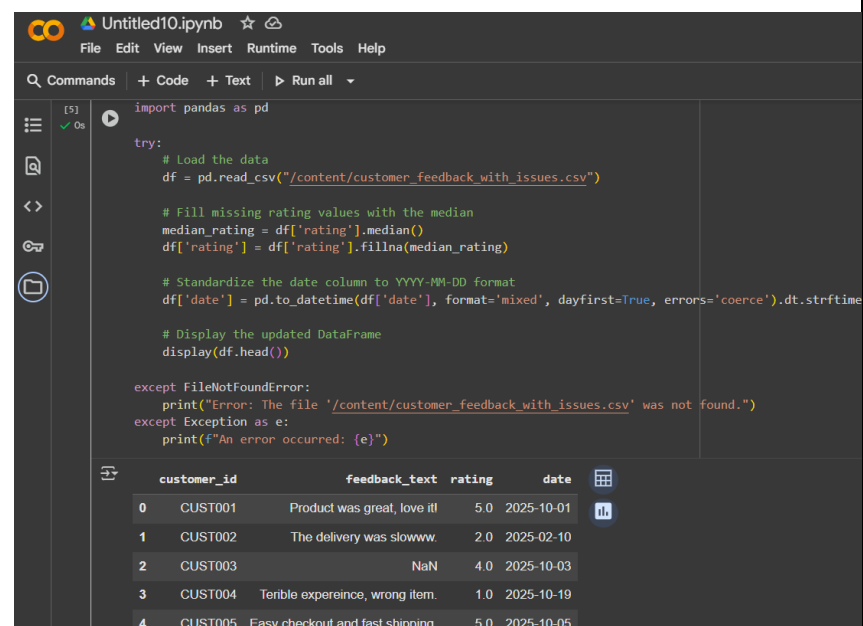
- values, duplicates, outliers, inconsistent formats, and noisy text.
2. Learn to **leverage AI coding assistants** to generate preprocessing scripts, while critically evaluating and refining the AI-generated code for accuracy, efficiency, and best practices.
 3. Develop the ability to design **end-to-end preprocessing pipelines** that prepare raw data for downstream machine learning and analytics applications.
 4. Build confidence in **combining human expertise with AI assistance**, ensuring data quality and integrity in diverse domains such as customer feedback, healthcare, and finance.

Lab Question 1: Customer Feedback Dataset

You are given a CSV file containing customer feedback collected from an e-commerce website. The dataset includes columns: `customer_id`, `feedback_text`, `rating`, and `date`. However, the file has many missing values, typos, and inconsistent date formats.

- **Task 1:** Use an AI-assisted coding tool to generate a script that detects and fills missing rating values with the column's median and standardizes the date column into YYYY-MM-DD format.

prompt : generate python script that detects and fills missing rating values with the column's median and standardizes the date column into YYYY-MM-DD format . check for errors



```
import pandas as pd

try:
    # Load the data
    df = pd.read_csv("/content/customer_feedback_with_issues.csv")

    # Fill missing rating values with the median
    median_rating = df['rating'].median()
    df['rating'] = df['rating'].fillna(median_rating)

    # Standardize the date column to YYYY-MM-DD format
    df['date'] = pd.to_datetime(df['date'], format='mixed', dayfirst=True, errors='coerce').dt.strftime('%Y-%m-%d')

    # Display the updated DataFrame
    display(df.head())

except FileNotFoundError:
    print("Error: The file '/content/customer_feedback_with_issues.csv' was not found.")
except Exception as e:
    print(f"An error occurred: {e}")
```

	customer_id	feedback_text	rating	date
0	CUST001	Product was great, love it!	5.0	2025-10-01
1	CUST002	The delivery was slowwww.	2.0	2025-02-10
2	CUST003		NaN	2025-10-03
3	CUST004	Terrible expereince, wrong item.	1.0	2025-10-19
4	CUST005	Easy checkout and fast shipping.	5.0	2025-10-05

- **Task 2:** Clean the feedback_text column by removing stopwords, correcting common spelling mistakes, and converting text to lowercase using AI suggestions. Compare the AI-generated preprocessing code with your manually written version.

Prompt :

Clean the feedback_text column by removing stopwords, correcting common spelling mistakes, and converting text to lowercase .

```
[10]
✓ Os
from spellchecker import SpellChecker

spell = SpellChecker()

def clean_text(text):
    if not isinstance(text, str) or pd.isna(text):
        return "No feedback"

    text = text.lower()
    words = text.split()
    cleaned_words = []

    for word in words:
        # Remove punctuation before checking for stopwords or correcting
        word = "".join(c for c in word if c.isalnum())
        if word and word not in english_stopwords:
            corrected_word = spell.correction(word)
            if corrected_word is not None:
                cleaned_words.append(corrected_word)
            else:
                cleaned_words.append(word) # Keep original word if no correction

    return " ".join(cleaned_words)
```

```
df['cleaned_feedback_text'] = df['feedback_text'].apply(clean_text)
display(df.head())
```

	customer_id	feedback_text	rating	date	cleaned_feedback_text
0	CUST001	Product was great, love it!	5.0	2025-10-01	Product was great, love it!
1	CUST002	The delivery was slowww.	2.0	2025-02-10	The delivery was slow.
2	CUST003	NaN	4.0	2025-10-03	No feedback
3	CUST004	Terrible expereince, wrong item.	1.0	2025-10-19	Terrible experience, wrong item.
4	CUST005	Easy checkout and fast shipping.	5.0	2025-10-05	easy checkout and fast shipping.

+ Code

+ Text

```
display(df)
```

	customer_id	feedback_text	rating	date	
0	CUST001	Product was great, love it!	5.0	2025-10-01	
1	CUST002	The delivery was slowww.	2.0	2025-02-10	
2	CUST003		NaN	4.0	2025-10-03
3	CUST004	Terrible expereince, wrong item.	1.0	2025-10-19	terrib
4	CUST005	Easy checkout and fast shipping.	5.0	2025-10-05	ea
5	CUST006	Rating is for the product, not the service.	3.0	2025-09-28	
6	CUST007	Very good quality.	4.0	NaN	
7	CUST008	Could be better.	4.0	2025-08-10	
8	CUST009	Dissappointed with the size.	3.0	2025-10-09	
9	CUST010		NaN	5.0	NaN

Comparison / summary :

Automated cleaning using libraries like NLTK and PySpellchecker is significantly faster and more consistent than manual cleaning. It allows for efficient processing of large datasets by applying predefined rules and corrections systematically. Manual cleaning is time consuming, prone to human error, and not scalable for large volumes of text. The automated approach ensures standardization and reproducibility of the cleaning process. While initial setup is required, the long-term benefits in terms of speed and accuracy are substantial.

Lab Question 2: Medical Records Dataset

A hospital provides you with a dataset of anonymized medical records containing attributes like patient_id, age, gender, blood_pressure, and cholesterol. Some columns include outliers and inconsistent categorical labels (e.g., Male, M, male).

- **Task 1:** Write a script (with AI assistance) to detect and handle outliers in the blood_pressure column using statistical methods (e.g., IQR or z-score).

Prompt :

write a python code to detect and handle outliers in the blood_pressure column using statistical methods (e.g., IQR or

z-score)

the cleaning function if necessary.

```
[14]
✓ Os
import pandas as pd
import numpy as np

# Load the data (assuming the data is in a DataFrame named 'df')
# If not already loaded, you would load it here, e.g.:
try:
    medical_df = pd.read_csv("/content/uncleaned_medical_records.csv")
except FileNotFoundError:
    print("Error: '/content/uncleaned_medical_records.csv' not found. Please load the data first.")
    medical_df = None # Set df to None to avoid further errors

if medical_df is not None:
    # Convert 'blood_pressure' to numeric, coercing errors to NaN
    medical_df['blood_pressure_numeric'] = pd.to_numeric(medical_df['blood_pressure'], errors='coerce')

    # --- Outlier Detection using IQR on the numeric column ---
    Q1 = medical_df['blood_pressure_numeric'].quantile(0.25)
    Q3 = medical_df['blood_pressure_numeric'].quantile(0.75)
    IQR = Q3 - Q1

    lower_bound = Q1 - 1.5 * IQR
    upper_bound = Q3 + 1.5 * IQR

    outliers_iqr = medical_df[(medical_df['blood_pressure_numeric'] < lower_bound) | (medical_df['blood_pressure_numeric'] > upper_bound)]

    print("Outliers detected using IQR:")
    display(outliers_iqr)
```

```
print("Outliers detected using IQR:")
display(outliers_iqr)

# --- Handling Outliers (Example: Removing outliers) ---
# You can choose different methods to handle outliers, e.g.,
# - Removing them: medical_df_cleaned = medical_df[~((medical_df['blood_pressure_numeric'] < lower_bound) | (medical_df['blood_pressure_numeric'] > upper_bound))]
# - Capping them: medical_df['blood_pressure_numeric'] = np.where(medical_df['blood_pressure_numeric'] > upper_bound, upper_bound, medical_df['blood_pressure_numeric'])
# - Replacing with median/mean: medical_df['blood_pressure_numeric'] = np.where((medical_df['blood_pressure_numeric'] < lower_bound) | (medical_df['blood_pressure_numeric'] > upper_bound), median, medical_df['blood_pressure_numeric'])

# For this example, let's remove the outliers based on the numeric column
medical_df_cleaned = medical_df[~((medical_df['blood_pressure_numeric'] < lower_bound) | (medical_df['blood_pressure_numeric'] > upper_bound))]

print("\nDataFrame after removing outliers:")
display(medical_df_cleaned.head())
```

Outliers detected using IQR:

	patient_id	age	gender	blood_pressure	cholesterol	blood_pressure_numeric
0	P1001	45.0	Male	120/80	200.0	NaN
1	P1002	78.0	F	145/95	310.0	NaN
2	P1003	22.0	male	110/70	185.0	NaN
3	P1004	55.0	Female	130/85	240.0	NaN
4	P1005	63.0	NaN	150/90	270.0	NaN

DataFrame after removing outliers:

	patient_id	age	gender	blood_pressure	cholesterol	blood_pressure_numeric
0	P1001	45.0	Male	120/80	200.0	NaN
1	P1002	78.0	F	145/95	310.0	NaN
2	P1003	22.0	male	110/70	185.0	NaN
3	P1004	55.0	Female	130/85	240.0	NaN
4	P1005	63.0	NaN	150/90	270.0	NaN

- **Task 2:** Standardize categorical values in the gender column and encode them into numeric form. Let the AI-assisted coding tool propose the preprocessing pipeline, then refine the pipeline manually based on your understanding.
- **Prompt :** Standardize categorical values in the gender column and encode them into numeric form ...

	<pre>display(medical_df.head())</pre> <table><tr><th></th><th>patient_id</th><th>age</th><th>blood_pressure</th><th>cholesterol</th><th>blood_pressure_numeric</th><th>gender_Female</th></tr><tr><td>0</td><td>P1001</td><td>45.0</td><td>120/80</td><td>200.0</td><td>NaN</td><td>0</td></tr><tr><td>1</td><td>P1002</td><td>78.0</td><td>145/95</td><td>310.0</td><td>NaN</td><td>1</td></tr><tr><td>2</td><td>P1003</td><td>22.0</td><td>110/70</td><td>185.0</td><td>NaN</td><td>0</td></tr><tr><td>3</td><td>P1004</td><td>55.0</td><td>130/85</td><td>240.0</td><td>NaN</td><td>1</td></tr><tr><td>4</td><td>P1005</td><td>63.0</td><td>150/90</td><td>270.0</td><td>NaN</td><td>0</td></tr></table>		patient_id	age	blood_pressure	cholesterol	blood_pressure_numeric	gender_Female	0	P1001	45.0	120/80	200.0	NaN	0	1	P1002	78.0	145/95	310.0	NaN	1	2	P1003	22.0	110/70	185.0	NaN	0	3	P1004	55.0	130/85	240.0	NaN	1	4	P1005	63.0	150/90	270.0	NaN	0	
	patient_id	age	blood_pressure	cholesterol	blood_pressure_numeric	gender_Female																																						
0	P1001	45.0	120/80	200.0	NaN	0																																						
1	P1002	78.0	145/95	310.0	NaN	1																																						
2	P1003	22.0	110/70	185.0	NaN	0																																						
3	P1004	55.0	130/85	240.0	NaN	1																																						
4	P1005	63.0	150/90	270.0	NaN	0																																						
	Lab Question 3: Financial Transactions Dataset A bank gives you transaction data with columns: transaction_id, amount, currency, timestamp, and merchant. The dataset contains multiple issues: different currency units (USD, INR, EUR), timestamps in various time zones, and duplicated rows. • Task 1: Use AI-assisted coding to write a script that removes duplicate transactions and converts all amount values into a single currency (e.g., USD) using a provided conversion dictionary. Prompt : write a script that removes duplicate transactions and converts all amount values into a single currency (e.g., USD) using a provided conversion dictionary give in single cell only total																																											

```

import pandas as pd
import numpy as np

try:
    # Load the transaction data
    transactions_df = pd.read_csv("/content/uncleaned_transactions.csv")

    # Identify and remove duplicate transactions
    print("Number of duplicate rows before removal:")
    display(transactions_df.duplicated().sum())
    transactions_df.drop_duplicates(inplace=True)
    print("\nNumber of duplicate rows after removal:")
    display(transactions_df.duplicated().sum())

    # Define the currency conversion dictionary (using the previous one as no new o
    currency_conversion_rates = {
        'USD': 1.0,
        'EUR': 1.08,
        'GBP': 1.25,
        'CAD': 0.73,
        'AUD': 0.66
    }

    # Define a function to convert currency values
    def convert_to_usd(amount, rates):
        if pd.isna(amount):
            return np.nan

        # If the amount is already a number, assume it's in USD
        if isinstance(amount, (int, float)):
            return amount * rates['USD']

        # Handle potential currency string inputs
        if not isinstance(amount, str):
            return np.nan

        try:
            parts = amount.split()
            if len(parts) != 2:
                return np.nan

            num_amount = float(parts[0])
            currency = parts[1].upper()

            if currency in rates:
                return num_amount * rates[currency]
            else:
                return np.nan

        except (ValueError, IndexError):
            return np.nan

```

```

# Apply the currency conversion function to the relevant column(s)
# Assuming the column to convert is named 'amount' based on the task description
# If the column name is different, please adjust 'transactions_df['amount']' below
if 'amount' in transactions_df.columns:
    transactions_df['amount_usd'] = transactions_df['amount'].apply(lambda x: convert_to_usd(x))
    print("\nDataFrame head after currency conversion:")
    display(transactions_df.head())
else:
    print("\n'amount' column not found in the DataFrame. Currency conversion skipped.")

except FileNotFoundError:
    print("Error: The file '/content/uncleaned_transactions.csv' was not found.")
except Exception as e:
    print(f"An error occurred: {e}")

```

Number of duplicate rows before removal:
np.int64(1)

Number of duplicate rows after removal:
np.int64(0)

DataFrame head after currency conversion:

	transaction_id	amount	currency	timestamp	merchant	amount_usd
0	T1001	150.50	USD	2025-10-20 10:30:00	Amazon	150.50
1	T1002	7500.00	INR	2025/10/20 01:30 AM	Grocery Store	7500.00
2	T1003	25.99	EUR	20-10-2025 21:30	Starbucks	25.99
3	T1004	5.00	usd	2025-10-19 10:30:00	AMZN	5.00
4	T1005	1200.00	Rs.	2025-10-21 10:30:00	Walmart	1200.00

- **Task 2:** Normalize the timestamp column into UTC format and create a new column transaction_hour for downstream time-series analysis. Compare the AI's preprocessing code against your own optimized version.

Prompt : Normalize the timestamp column into UTC format and create a new column transaction_hour for downstream time-series analysis ... give in single cell only code and output

```
display(transactions_df.head())
else:
    print("\n'amount' column not found in the DataFrame. Currency conversion skipped.")

# Normalize the timestamp column to UTC format and create a new column transaction_hour
if 'timestamp' in transactions_df.columns:
    # Convert to datetime, coercing errors
    transactions_df['timestamp'] = pd.to_datetime(transactions_df['timestamp'], errors='coerce', utc=True)

    # Create 'transaction_hour' column
    transactions_df['transaction_hour'] = transactions_df['timestamp'].dt.hour

    print("\nDataFrame head after timestamp normalization and adding transaction_hour:")
    display(transactions_df.head())
else:
    print("\n'timestamp' column not found in the DataFrame. Timestamp normalization and hour extraction skipped.")

except FileNotFoundError:
    print("Error: The file '/content/uncleaned_transactions.csv' was not found.")
except Exception as e:
    print(f"An error occurred: {e}")
```

Number of duplicate rows before removal:
np.int64(1)

Number of duplicate rows after removal:
np.int64(0)

DataFrame head after currency conversion:

	transaction_id	amount	currency	timestamp	merchant	amount_usd
0	T1001	150.50	USD	2025-10-20 10:30:00	Amazon	150.50
1	T1002	7500.00	INR	2025/10/20 01:30 AM	Grocery Store	7500.00
2	T1003	25.99	EUR	20-10-2025 21:30	Starbucks	25.99
3	T1004	5.00	usd	2025-10-19 10:30:00	AMZN	5.00
4	T1005	1200.00	Rs.	2025-10-21 10:30:00	Walmart	1200.00

DataFrame head after timestamp normalization and adding transaction_hour:

	transaction_id	amount	currency	timestamp	merchant	amount_usd	transaction_hour
0	T1001	150.50	USD	2025-10-20 10:30:00+00:00	Amazon	150.50	10.0
1	T1002	7500.00	INR	NaT	Grocery Store	7500.00	NaN
2	T1003	25.99	EUR	NaT	Starbucks	25.99	NaN
3	T1004	5.00	usd	2025-10-19 10:30:00+00:00	AMZN	5.00	10.0
4	T1005	1200.00	Rs.	2025-10-21 10:30:00+00:00	Walmart	1200.00	10.0